

SALUS SECURITY

AUG 2026



CODE SECURITY ASSESSMENT

GAIN

Overview

Project Summary

- Name: Gain
- Platform: EVM-compatible chains
- Language: Solidity
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	Gain
Version	v2
Type	Solidity
Dates	Aug 18 2026
Logs	Aug 13 2026, Aug 18 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	4
Total Low-Severity issues	1
Total informational issues	1
Total	6

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Users can bypass the waiting time via early request	6
2. Users can bypass blacklist blocking	7
3. Improper share calculation for staking	8
4. Centralization risk	9
5. Missing token normalization in stake	10
2.3 Informational Findings	11
6. Redundant code	11
Appendix	12
Appendix 1 - Files in Scope	12

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Users can bypass the waiting time via early request	Medium	Business Logic	Resolved
2	Users can bypass blacklist blocking	Medium	Business Logic	Resolved
3	Improper share calculation for staking	Medium	Business Logic	Acknowledged
4	Centralization risk	Medium	Centralization	Acknowledged
5	Missing token normalization in stake	Low	Business Logic	Resolved
7	Redundant code	Informational	Configuration	Resolved

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Users can bypass the waiting time via early request

Severity: Medium

Category: Business Logic

Target:

- Gain/src/Gain.sol

Description

In Gain, users need to withdraw their assets via two steps, request claim and final claim. Users need to request withdrawal at first. Then users can claim the final withdrawal after the waiting time.

The problem here is that when users claim assets, the asset calculation will use the in-time share price. This will cause users to bypass the waiting time via one early withdrawal request. Users can always request a claim once they stake. After one month, if this user wants to withdraw assets, this user can claim directly and take no loss because this calculation uses the share price when users claim assets.

Gain/src/Gain.sol:L105-L116

```
function claim(uint256 claimId) external nonReentrant whenNotPaused onlyNotBlackList
onlyClaimEnable {
    ClaimItem storage claimItem = claimQueue[claimId];
    AssetsInfo storage assetsInfo = userAssetsInfo[msg.sender];
    require(!claimItem.isDone, AlreadyClaimed());
    // users can claim assets via this.
    require(block.timestamp >= claimItem.claimTime, ClaimWaiting());
    require(claimItem.user == msg.sender, InvalidSender());
    // Note: The final price will be determined by currenty share price. Not the share
    price in the redeem request.
    uint256 currentAmount = convertToAssets(claimItem.shareAmount);
}
```

Recommendation

When users request withdrawal, the related share should stop earning profit.

Status

This issue has been resolved by the team.

2. Users can bypass blacklist blocking

Severity: Medium

Category: Business Logic

Target:

- Gain/src/GainToken.sol

Description

In Gain, the admin can set one malicious user into the blacklist. Users in the blacklist are not allowed to request a claim or claim.

The problem here is that when one user is added into the blacklist, users can transfer their owned GainToken to another user and withdraw assets via request claim or flash withdrawal.

Gain/src/GainToken.sol:L6-L37

```
contract GainToken is ERC20, Ownable {
    address public gain;

    event UpdateGain(address _oldGain, address _newGain);
    modifier onlyGain(){
        require(msg.sender == gain);
        _;
    }
    ...
}
```

Recommendation

Block GainToken's transfer when users are in the blacklist.

Status

This issue has been resolved by the team.

3. Improper share calculation for staking

Severity: Medium

Category: Business Logic

Target:

- Gain/src/Gain.sol

Description

In Gain, the bot will update nav periodically. The bot will fetch the latest value and calculate nav according to the profit.

The problem here is that when users stake assets, the share price will use current nav. This will cause that the actual total share's value will exceed total balance.

For example: On Day 1, users deposit 1000 USD. The admin transfers assets to invest and earn 10 USD. The nav should be increased from $1e18$ to $1.01 * 1e18$. Before the nav is updated, another user stakes 10000 USD with current nav price $1e18$. Then after the nav is updated, users are expected to withdraw $11000 * 1.01$ USD. But actually, the protocol owns only 11010 USD.

Gain/src/Gain.sol:L347-L363

```
function stake(address underlyingToken, uint256 stakeAmount)
    external
    nonReentrant
    whenNotPaused
    onlyNotBlackList // users cannot stake for others.
    onlyStakeEnable // only stake enabled.
    onlySupportedToken(underlyingToken) // make sure that the input underlying token is
    supported.
{
    require(stakeAmount > 0, InvalidAmount());
    AssetsInfo storage assetsInfo = userAssetsInfo[msg.sender];
    // transfer underlying asset to this contract.
    IERC20(underlyingToken).safeTransferFrom(msg.sender, address(this), stakeAmount);
    uint256 shareAmount = convertToShare(stakeAmount);
    ...
}
```

Recommendation

Suggest using async deposit.

Status

This issue has been acknowledged by the team.

4. Centralization risk

Severity: Medium

Category: Centralization

Target:

- Gain/src/Gain.sol

Description

In Gain contract, there exists some privileged roles, for example `DEFAULT\_ADMIN\_ROLE`. These roles have the authority to execute some key functions such as `setSupportedToken`, `setTreasury`, etc.

If these roles' private keys are compromised, an attacker could trigger these functions to harm the whole protocol.

Gain/src/Gain.sol:L560-L566

```
function setTreasury(address _newTreasury) external onlyRole(DEFAULT_ADMIN_ROLE) {  
    require(_newTreasury != address(0), ZeroAddress());  
    require(_newTreasury != treasury, ConsistentValue());  
    address oldTreasury = treasury;  
    treasury = _newTreasury;  
    emit UpdateTreasury(oldTreasury, _newTreasury);  
}
```

Recommendation

We recommend transferring privileged accounts to multi-sig accounts with timelock governors for enhanced security. This ensures that no single person has full control over the accounts and that any changes must be authorized by multiple parties.

Status

This issue has been acknowledged by the team.

5. Missing token normalization in stake

Severity: Low

Category: Business Logic

Target:

- Gain/src/Gain.sol

Description

In Gain, the contract may support multiple underlying tokens. Users can choose one supported underlying token to stake and mint gain share token.

In the stake function, the input stakeAmount will be used to calculate the share amount directly. If the admin adds supported tokens, with different decimals or different prices, this will cause the incorrect share calculation.

Gain/src/Gain.sol:L347-L363

```
function stake(address underlyingToken, uint256 stakeAmount)
    external
    nonReentrant
    whenNotPaused
    onlyNotBlackList // users cannot stake for others.
    onlyStakeEnable // only stake enabled.
    onlySupportedToken(underlyingToken)
{
    require(stakeAmount > 0, InvalidAmount());
    AssetsInfo storage assetsInfo = userAssetsInfo[msg.sender];
    // transfer underlying asset to this contract.
    IERC20(underlyingToken).safeTransferFrom(msg.sender, address(this), stakeAmount);
    uint256 shareAmount = convertToShare(stakeAmount);
}
```

Recommendation

Normalize the token amount or make sure that supported tokens share the same decimal and price.

Status

This issue has been resolved by the team.

2.3 Informational Findings

6. Redundant code

Severity: Informational

Category: Redundancy

Target:

- Gain/src/Gain.sol

Description

In Gain contract, there are some events, errors which are not used in this contract, including error StakeAmountOutOfRange, error UnsupportedTicket, and also the event Mint.

Gain/src/Gain.sol:L18-L277

```
contract Gain is AccessControl, Pausable, ReentrancyGuard {  
    event Mint(address to, uint256 mintAmount);  
    error StakeAmountOutOfRange();  
    error UnsupportedTicket();  
    ...  
}
```

Recommendation

Remove the redundant code.

Status

This issue has been resolved by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files:

File	SHA-1 hash
Gain.sol	3e6625872a76317067bb691d48b86b17323660d4
GainToken.sol	8cf2ce84c02d4eb4a7e81aeaa806e21d96c5b474